

Lab 2.1: C Libraries, Linking, and ELF Executable Structure

Introduction

This lab follows a small C program from source code to a running executable. It shows how the program links to the C Standard Library (libc), where that library lives on disk, how static and dynamic linking differ, and how the resulting ELF file can be inspected.

The lab answers three questions:

1. How does a C program link to the C Standard Library?
2. Where do C library headers and compiled code physically live on disk?
3. What is the difference between static and dynamic linking, and how can the resulting executables be inspected?

Lab environment

Item	Used in this lab
Operating system	Ubuntu Linux in a Docker container (<code>oslab</code>)
Compiler	<code>gcc</code> (<code>build-essential</code>)
Inspection tools	<code>readelf</code> , <code>ldd</code> , <code>ls</code> , <code>grep</code>
Working folder	<code>/labs/lab2.1</code>

The program: `procinfo.c`

The program prints its own process ID (24), its parent's ID (1), the current time and its executable path, using six functions that all come from the C library.

Function	Header	Purpose
<code>getpid()</code> , <code>getppid()</code>	<code><unistd.h></code>	Process ID and parent process ID
<code>time()</code> , <code>localtime()</code>	<code><time.h></code>	Current time, converted to a date structure
<code>printf()</code>	<code><stdio.h></code>	Formatted output
<code>asctime()</code>	<code><time.h></code>	Turns the date structure into text

Source code

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>

int main(void) {
    // Get current process ID
    pid_t my_pid = getpid();
    pid_t my_ppid = getppid();

    // Get current time
    time_t current_time = time(NULL);
    struct tm *time_info = localtime(&current_time);

    // Format and display information
    printf("=== Process Information===\n");
    printf("My Process ID (PID): %d\n", my_pid);
    printf("Parent Process ID: %d\n", my_ppid);
    printf("Current Time: %s", asctime(time_info));
    printf("Executable Path:/proc/self/exe\n");

    return 0;
}
```

```

GNU nano 7.2 procinfo.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>
int main(void) {
    // Get current process ID
    pid_t my_pid = getpid();
    pid_t my_ppid = getppid();
    // Get current time
    time_t current_time = time(NULL);
    struct tm *time_info =
    localtime(&current_time);
    // Format and display information
    printf("=== Process Information===\n");
    printf("My Process ID (PID): %d\n", my_pid);
    printf("Parent Process ID: %d\n", my_ppid);
    printf("Current Time: %s", asctime(time_info));
    printf("Executable Path: /proc/self/exe\n");
    return 0;
}

```

Compile and run

```
gcc procinfo.c -o procinfo
./procinfo
```

```

root@9ed76ed0a3e1:/labs/lab2.1# gcc procinfo.c -o procinfo
root@9ed76ed0a3e1:/labs/lab2.1# ./procinfo
=== Process Information===
My Process ID (PID): 24
Parent Process ID: 1
Current Time: Thu Oct  8 03:58:34 2026
Executable Path: /proc/self/exe
root@9ed76ed0a3e1:/labs/lab2.1# |

```

Observations

- The parent PID is 1 because inside the Docker container the shell is the first process, PID 1.
- The time shows 03:58 because the container uses UTC; Nepal time is UTC+05:45, so it was 09:43 locally.

- When copying the code from the lab sheet, two `printf` strings were split across lines, causing "missing terminating " character" errors. Joining each string onto one line fixed it.

Where the C library lives: two halves

The C library is split into two physical parts: **header files** that only describe the functions, and **library files** that hold the real machine code.

Part	Location	Contains	Used by
Header files (<code>.h</code>)	<code>/usr/include/</code>	Declarations: function prototypes, macros, types	The compiler (preprocessor), at compile time
Static library (<code>.a</code>)	<code>/lib/x86_64-linux-gnu/libc.a</code>	Archive of object files, copied into the program	The linker, with <code>-static</code>
Shared library (<code>.so</code>)	<code>/lib/x86_64-linux-gnu/libc.so.6</code>	Compiled code loaded at run time, shared in RAM	The dynamic linker, when the program starts

Part 1: header files, the construction manual

```
ls /usr/include/stdio.h
ls /usr/include/unistd.h
grep "printf" /usr/include/stdio.h | head -2
grep "getpid" /usr/include/unistd.h | head -2
```

```
root@9ed76ed0a3e1:/# ls /usr/include/stdio.h
/usr/include/stdio.h
root@9ed76ed0a3e1:/# ls /usr/include/unistd.h
/usr/include/unistd.h
root@9ed76ed0a3e1:/# grep "printf" /usr/include/stdio.h | head -2
/* Maximum length of printf output for a NaN.  */
extern int fprintf (FILE *__restrict __stream,
root@9ed76ed0a3e1:/# grep "printf" /usr/include/unistd.h | head -2
root@9ed76ed0a3e1:/# grep "getpid" /usr/include/unistd.h | head -2
extern __pid_t getpid (void) __THROW;
root@9ed76ed0a3e1:/# |
```

The header contains only a declaration: `extern __pid_t getpid (void) __THROW;`. It tells the compiler that `getpid()` exists, takes no arguments and returns a process ID, but contains no code. `#include` copies this text into the program before compiling. (`grep "printf" onunistd.h` printed nothing because `printf` is declared in `stdio.h`, not `unistd.h`.)

Part 2: compiled libraries, the actual code

```
ls /lib/x86_64-linux-gnu/libc.so*
ls /lib/x86_64-linux-gnu/libc.a
```

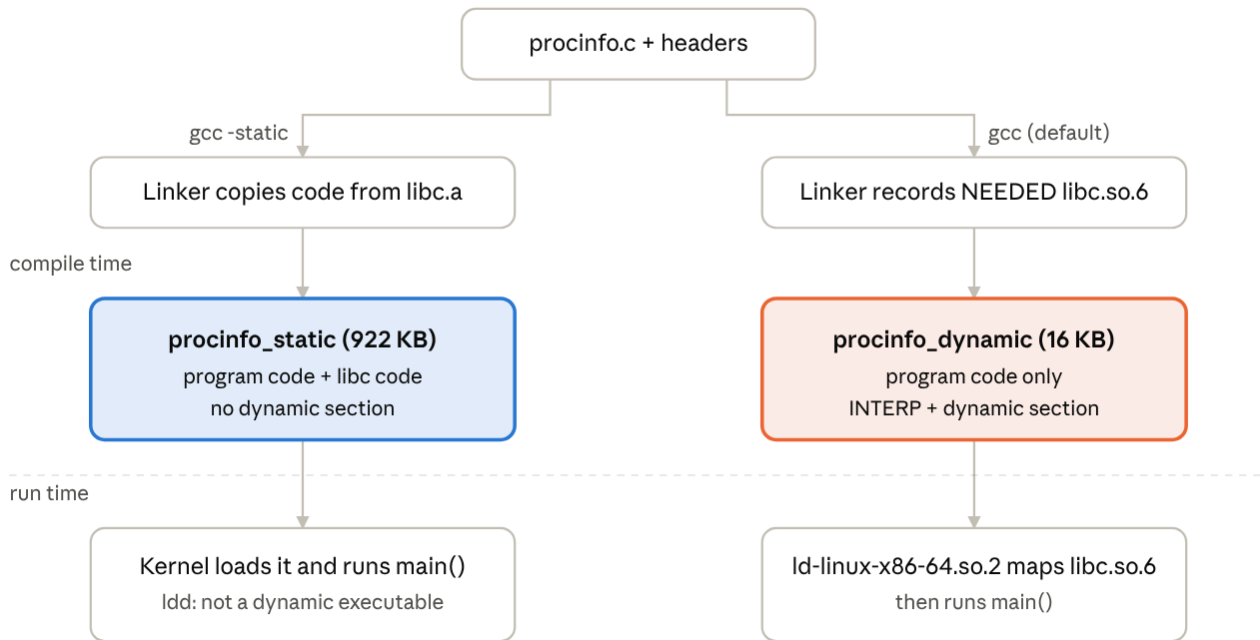
```
root@9ed76ed0a3e1:/# ls /lib/x86_64-linux-gnu/libc.so*
/lib/x86_64-linux-gnu/libc.so  /lib/x86_64-linux-gnu/libc.so.6
root@9ed76ed0a3e1:/# ls /lib/x86_64-linux-gnu/libc.a
/lib/x86_64-linux-gnu/libc.a
root@9ed76ed0a3e1:/# |
```

Both forms of libc are present: `libc.so.6` (shared, shown in green because it is executable) and `libc.a` (static archive). These files contain the real machine code for `printf()`, `getpid()` and the other functions.

Static vs dynamic linking

A statically linked program carries its own copy of the library code and is 922 KB; a dynamically linked one is only 16 KB because it records which library it needs and loads it at run time.

Static linking copies libc in; dynamic linking loads it at run time



Static vs dynamic linking · compile time and run time

	Static linking	Dynamic linking
Command	<code>gcc -static procinfo.c -o procinfo_static</code>	<code>gcc procinfo.c -o procinfo_dynamic</code>
Library used	<code>libc.a</code> , copied in at compile time	<code>libc.so.6</code> , loaded at run time
File size	Large (922 KB measured)	Small (16 KB measured)
Needs libc on the machine?	No, fully self-contained	Yes, fails with "shared library not found" if missing
Library updates	Program must be recompiled	Updating <code>libc.so</code> fixes every program at once
Memory	Each program has its own copy	One copy in RAM is shared by all programs
Typical use	Containers, embedded systems	Default on modern Linux

Static build

```
gcc -static procinfo.c -o procinfo_static
ls -lh procinfo_static
```

```
root@9ed76ed0a3e1:/labs/lab2.1# gcc -static procinfo.c -o procinfo_static
root@9ed76ed0a3e1:/labs/lab2.1# ls -lh procinfo_static
-rwxr-xr-x 1 root root 922K Oct  8 04:06 procinfo_static
root@9ed76ed0a3e1:/labs/lab2.1# |
```

Dynamic build

```
gcc procinfo.c -o procinfo_dynamic
ls -lh procinfo_dynamic
```

```
root@9ed76ed0a3e1:/labs/lab2.1# gcc procinfo.c -o procinfo_dynamic
root@9ed76ed0a3e1:/labs/lab2.1# ls -lh procinfo_dynamic procinfo_static
-rwxr-xr-x 1 root root 16K Oct  8 04:20 procinfo_dynamic
-rwxr-xr-x 1 root root 922K Oct  8 04:12 procinfo_static
```

The dynamic binary is **16 KB** against **922 KB** for the static one, about 58 times smaller, because it contains only the program's own code and relies on the shared `libc.so.6` at run time.

Issue encountered

The first attempt at the dynamic build accidentally used the `-static` flag, so `procinfo_dynamic` came out the same size as the static file:

```
root@9ed76ed0a3e1:/labs/lab2.1# gcc -static procinfo.c -o procinfo_dynamic
root@9ed76ed0a3e1:/labs/lab2.1# ls -lh procinfo_dynamic
-rwxr-xr-x 1 root root 922K Oct  8 04:08 procinfo_dynamic
root@9ed76ed0a3e1:/labs/lab2.1# |
```

This was noticed when `readelf -d procinfo_dynamic` reported "There is no dynamic section in this file." Rebuilding without `-static` fixed it. **Security note:** the size and missing dynamic section are a quick way to tell how a binary was linked, which is useful when analysing unknown executables.

ELF inspection

Both executables are 64-bit ELF files; only the dynamic one has an interpreter, a dynamic section and a dependency on `libc.so.6`.

ELF (Executable and Linkable Format) is the file format Linux uses for programs. It is made of an **ELF header** (global information), **program headers** (how to load the file into memory) and **section headers** (the code and data parts).

ELF header: `readelf -h``readelf -h procinfo_dynamic`

```

root@9ed76ed0a3e1:/labs/lab2.1# readelf -h procinfo_dynamic
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00
  Class:                               ELF64
  Data:                               2's complement, little endian
  Version:                             1 (current)
  OS/ABI:                              UNIX - System V
  ABI Version:                         0
  Type:                                DYN (Position-Independent Executable file)
  Machine:                             Advanced Micro Devices X86-64
  Version:                             0x1
  Entry point address:                 0x1140
  Start of program headers:            64 (bytes into file)
  Start of section headers:           14272 (bytes into file)
  Flags:                               0x0
  Size of this header:                 64 (bytes)
  Size of program headers:             56 (bytes)
  Number of program headers:           13
  Size of section headers:             64 (bytes)
  Number of section headers:           31
  Section header string table index:   30

```

Field	Value	Meaning
Magic	7f 45 4c 46	0x7f followed by "ELF" in ASCII; identifies the file as ELF
Class	ELF64	64-bit program
Data	2's complement, little endian	Byte order used by x86 CPUs
OS/ABI	UNIX - System V	Standard Linux ABI
Type	DYN (Position-Independent Executable file)	Can be loaded at any address, which allows ASLR
Machine	Advanced Micro Devices X86-64	Runs on 64-bit Intel/AMD processors
Entry point address	0x1140	Offset of the first instruction; the real address is chosen at load time
Number of program headers	13	Segments described for loading
Number of section headers	31	Sections such as <code>.text</code> , <code>.data</code> , <code>.dynamic</code>

Static vs dynamic ELF header. The same command on the static build gave a different picture:

Field	procinfo_dynamic	procinfo_static
Type	DYN (position-independent)	EXEC (fixed address)
OS/ABI	UNIX - System V	UNIX - GNU
Entry point	0x1140 (relative)	0x4018e0 (fixed)
Program headers	13	10
Section headers	31	28
Section headers start at	14,272 bytes	941,856 bytes

The section headers sit at the end of the file, so their offset also shows how much bigger the static file is.

```

root@9ed76ed0a3e1:/labs/lab2.1# readelf -h procinfo_dynamic
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 03 00 00 00 00 00 00 00 00
  Class:                               ELF64
  Data:                                   2's complement, little endian
  Version:                               1 (current)
  OS/ABI:                                UNIX - GNU
  ABI Version:                           0
  Type:                                  EXEC (Executable file)
  Machine:                               Advanced Micro Devices X86-64
  Version:                               0x1
  Entry point address:                   0x4018e0
  Start of program headers:               64 (bytes into file)
  Start of section headers:               941856 (bytes into file)
  Flags:                                  0x0
  Size of this header:                     64 (bytes)
  Size of program headers:                 56 (bytes)
  Number of program headers:                10
  Size of section headers:                 64 (bytes)
  Number of section headers:                28
  Section header string table index:       27
root@9ed76ed0a3e1:/labs/lab2.1# |

```

Program headers: `readelf -l`

```
readelf -l procinfo_dynamic
```

```

root@9ed76ed0a3e1:/labs/lab2.1# readelf -l procinfo_dynamic

Elf file type is DYN (Position-Independent Executable file)
Entry point 0x1140
There are 13 program headers, starting at offset 64

Program Headers:
  Type           Offset             VirtAddr           PhysAddr
                 FileSiz            MemSiz             Flags   Align
  PHDR           0x0000000000000040 0x0000000000000040 0x0000000000000040
                 0x00000000000002d8 0x00000000000002d8 R       0x8
  INTERP         0x0000000000000318 0x0000000000000318 0x0000000000000318
                 0x000000000000001c 0x000000000000001c R       0x1
    [Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
  LOAD           0x0000000000000000 0x0000000000000000 0x0000000000000000
                 0x00000000000007d8 0x00000000000007d8 R       0x1000
  LOAD           0x0000000000000100 0x0000000000000100 0x0000000000000100 0x0000000000000100
                 0x000000000000030d 0x000000000000030d R E     0x1000
  LOAD           0x0000000000000200 0x0000000000000200 0x0000000000000200
                 0x000000000000016c 0x000000000000016c R       0x1000
  LOAD           0x00000000000002d8 0x00000000000002d8 0x00000000000002d8
                 0x0000000000000298 0x0000000000000298 RW      0x1000
  DYNAMIC        0x00000000000002d9 0x00000000000002d9 0x00000000000002d9
                 0x00000000000001f0 0x00000000000001f0 RW      0x8
  NOTE           0x0000000000000338 0x0000000000000338 0x0000000000000338
                 0x0000000000000030 0x0000000000000030 R       0x8
  NOTE           0x0000000000000368 0x0000000000000368 0x0000000000000368
                 0x0000000000000044 0x0000000000000044 R       0x4
  GNU_PROPERTY   0x0000000000000338 0x0000000000000338 0x0000000000000338
                 0x0000000000000030 0x0000000000000030 R       0x8
  GNU_EH_FRAME   0x0000000000000208 0x0000000000000208 0x0000000000000208
                 0x0000000000000034 0x0000000000000034 R       0x4
  GNU_STACK      0x0000000000000000 0x0000000000000000 0x0000000000000000
                 0x0000000000000000 0x0000000000000000 RW      0x10
  GNU_RELRO      0x00000000000002d8 0x00000000000002d8 0x00000000000002d8
                 0x0000000000000280 0x0000000000000280 R       0x1

Section to Segment mapping:
Segment Sections...
00
01      .interp
02      .interp .note.gnu.property .note.gnu.build-id .note.ABI-tag .gnu.hash .dynsym .dynstr .gnu.v
ersion .gnu.version_r .rela.dyn .rela.plt
03      .init .plt .plt.got .plt.sec .text .fini
04      .rodata .eh_frame_hdr .eh_frame
05      .init_array .fini_array .dynamic .got .data .bss
06      .dynamic
07      .note.gnu.property
08      .note.gnu.build-id .note.ABI-tag
09      .note.gnu.property
10      .eh_frame_hdr
11
12      .init_array .fini_array .dynamic .got
root@9ed76ed0a3e1:/labs/lab2.1#

```

Segment	Flags	Meaning
PHDR	R	The program header table itself
INTERP	R	Requests the program interpreter <code>/lib64/ld-linux-x86-64.so.2</code> , the dynamic linker
LOAD (1st)	R	Read-only headers, symbol and relocation tables
LOAD (2nd)	R E	Program code (<code>.text</code> , <code>.plt</code>); readable and executable
LOAD (3rd)	R	Read-only data (<code>.rodata</code>) such as the <code>printf</code> strings

Segment	Flags	Meaning
LOAD (4th)	RW	Writable data (<code>.data</code> , <code>.bss</code> , <code>.got</code>)
DYNAMIC	RW	The dynamic section, read by the dynamic linker
GNU_EH_FRAME	R	Exception and stack-unwinding information
GNU_STACK	RW	The stack is not executable (NX), blocking injected stack code
GNU_RELRO	R	Data made read-only after start-up, so the GOT cannot be overwritten

No segment is both writable and executable (W^X), which makes code-injection attacks harder.

Static vs dynamic program headers. The static build has 10 headers and **no PHDR, INTERP or DYNAMIC** segments, because it needs no loader and no shared libraries. Instead it has a TLS segment for thread-local data that the dynamic build gets from libc at run time.

```
root@9ed76ed0a3e1:/labs/lab2.1# readelf -l procinfo_dynamic
```

```
Elf file type is EXEC (Executable file)
```

```
Entry point 0x4018e0
```

```
There are 10 program headers, starting at offset 64
```

```
Program Headers:
```

Type	Offset FileSiz	VirtAddr MemSiz	PhysAddr Flags Align
LOAD	0x0000000000000000	0x0000000000400000	0x0000000000400000
	0x0000000000000510	0x0000000000000510	R 0x1000
LOAD	0x0000000000001000	0x0000000000401000	0x0000000000401000
	0x000000000000a0b1d	0x000000000000a0b1d	R E 0x1000
LOAD	0x000000000000a2000	0x00000000004a2000	0x00000000004a2000
	0x000000000000274e5	0x000000000000274e5	R 0x1000
LOAD	0x000000000000c9f50	0x00000000004caf50	0x00000000004caf50
	0x00000000000005b90	0x000000000000b4f8	RW 0x1000
NOTE	0x00000000000000270	0x0000000000400270	0x0000000000400270
	0x00000000000000030	0x00000000000000030	R 0x8
NOTE	0x000000000000002a0	0x00000000004002a0	0x00000000004002a0
	0x00000000000000044	0x00000000000000044	R 0x4
TLS	0x000000000000c9f50	0x00000000004caf50	0x00000000004caf50
	0x00000000000000018	0x00000000000000058	R 0x8
GNU_PROPERTY	0x00000000000000270	0x0000000000400270	0x0000000000400270
	0x00000000000000030	0x00000000000000030	R 0x8
GNU_STACK	0x00000000000000000	0x00000000000000000	0x00000000000000000
	0x00000000000000000	0x00000000000000000	RW 0x10
GNU_RELRO	0x000000000000c9f50	0x00000000004caf50	0x00000000004caf50
	0x00000000000040b0	0x00000000000040b0	R 0x1

```
Section to Segment mapping:
```

```
Segment Sections...
```

00	.note.gnu.property .note.gnu.build-id .note.ABI-tag .rela.plt
01	.init .plt .text .fini
02	.rodata .stapsdt.base rodata.cst32 .eh_frame .gcc_except_table
03	.tdata .init_array .fini_array .data.rel.ro .got .got.plt .data .bss
04	.note.gnu.property
05	.note.gnu.build-id .note.ABI-tag
06	.tdata .tbss
07	.note.gnu.property
08	
09	.tdata .init_array .fini_array .data.rel.ro .got

```
root@9ed76ed0a3e1:/labs/lab2.1# |
```

Dynamic section: `readelf -d`

```

root@9ed76ed0a3e1:/labs/lab2.1# readelf -d procinfo_dynamic

Dynamic section at offset 0x2d90 contains 27 entries:
  Tag                Type                               Name/Value
 0x0000000000000001 (NEEDED)                             Shared library: [libc.so.6]
 0x000000000000000c (INIT)                               0x1000
 0x000000000000000d (FINI)                               0x1300
 0x0000000000000019 (INIT_ARRAY)                             0x3d80
 0x000000000000001b (INIT_ARRAYSZ)                             8 (bytes)
 0x000000000000001a (FINI_ARRAY)                             0x3d88
 0x000000000000001c (FINI_ARRAYSZ)                             8 (bytes)
 0x0000000006ffffef5 (GNU_HASH)                             0x3b0
 0x0000000000000005 (STRTAB)                               0x528
 0x0000000000000006 (SYMTAB)                               0x3d8
 0x000000000000000a (STRSZ)                               208 (bytes)
 0x000000000000000b (SYMENT)                               24 (bytes)
 0x0000000000000015 (DEBUG)                               0x0
 0x0000000000000003 (PLTGOT)                               0x3f80
 0x0000000000000002 (PLTRELSZ)                             192 (bytes)
 0x0000000000000014 (PLTREL)                               RELA
 0x0000000000000017 (JMPREL)                               0x718
 0x0000000000000007 (RELA)                               0x658
 0x0000000000000008 (RELASZ)                               192 (bytes)
 0x0000000000000009 (RELAENT)                              24 (bytes)
 0x000000000000001e (FLAGS)                               BIND_NOW
 0x0000000006fffffb (FLAGS_1)                             Flags: NOW PIE
 0x0000000006fffffe (VERNEED)                             0x618
 0x0000000006ffffff (VERNEEDNUM)                           1
 0x0000000006fffff0 (VERSYM)                               0x5f8
 0x0000000006fffff9 (RELACOUNT)                             3
 0x0000000000000000 (NULL)                                0x0
root@9ed76ed0a3e1:/labs/lab2.1# |

```

After rebuilding, `procinfo_dynamic` has a dynamic section with 27 entries. The key lines are:

- `(NEEDED) Shared library: [libc.so.6]`: the only library the program depends on.
- `(FLAGS_1) Flags: NOW PIE`: a position-independent executable whose library symbols are all resolved at start-up (`BIND_NOW`). PIE lets the OS load the program at a random address (ASLR), and `BIND_NOW` enables full RELRO; both make attacks harder.

On the static binary, the same command prints "There is no dynamic section in this file."

Library dependencies: `ldd`

```
root@9ed76ed0a3e1:/labs/lab2.1# ldd procinfo_dynamic
linux-vdso.so.1 (0x00007fffd534e000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f7db9d8c000)
/lib64/ld-linux-x86-64.so.2 (0x00007f7db9fa9000)
root@9ed76ed0a3e1:/labs/lab2.1# |
```

Dependency	Meaning
<code>linux-vdso.so.1</code>	Virtual library provided by the kernel for fast system calls such as <code>time()</code>
<code>libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6</code>	The C library, found at the path checked earlier
<code>/lib64/ld-linux-x86-64.so.2</code>	The dynamic linker that loads the libraries when the program starts

```
/lib64/ld-linux-x86-64.so.2 (0x00007f7db9fa9000)
root@9ed76ed0a3e1:/labs/lab2.1# ldd procinfo_static
not a dynamic executable
root@9ed76ed0a3e1:/labs/lab2.1# |
```

The static binary has no dependencies, so `ldd` reports "not a dynamic executable".

Knowledge check answers

1. What is the fundamental difference between a C header file and a shared library?

A header file (`.h`, in `/usr/include/`) contains only **declarations**: function prototypes, macros and types. It tells the compiler what a function looks like, for example `extern __pid_t getpid (void);`, but has no executable code. A shared library (`.so`, such as `/lib/x86_64-linux-gnu/libc.so.6`) contains the **compiled machine code** that actually runs. The header is the construction manual; the library is the bricks and steel. Headers are used at compile time, shared libraries at run time.

2. What is static and dynamic linking?

- **Static linking** (`gcc -static`) copies the needed library code from `libc.a` into the executable at compile time. The result is self-contained but large (922 KB in this lab), and must be recompiled to get library updates.
- **Dynamic linking** (the default) stores only references to shared libraries. The dynamic linker `/lib64/ld-linux-x86-64.so.2` loads `libc.so.6` into memory when the program starts. The file is small, library code is shared in RAM, and updates apply to every program, but the program fails if the library is missing.

3. What is the ELF header? Commands to check headers.

The ELF header is the first 64 bytes of an ELF file. It holds the file's global information: the magic number `7f 45 4c 46` that marks it as ELF, the class (32 or 64-bit), byte order, file type (EXEC, DYN or REL), target machine, entry point address, and where the program and section headers start.

Command	Shows
<code>readelf -h <file></code>	ELF header
<code>readelf -l <file></code>	Program headers (segments and permissions)
<code>readelf -S <file></code>	Section headers (<code>.text</code> , <code>.data</code> , ...)
<code>readelf -d <file></code>	Dynamic section (needed libraries)
<code>file <file></code>	Quick summary: ELF type, architecture, static or dynamic

4. What does bookmark mean? Provide an example.

A bookmark is a saved shortcut that lets you return quickly to a location without finding it again. In the Linux shell, a common way to bookmark a working folder is an alias:

```
alias lab='cd /labs/lab2.1'
lab          # jumps straight to the lab folder
```

In a PDF or document, a bookmark is a clickable outline entry that jumps to a section, such as "ELF inspection" in this report.

5. Documentation and GitHub

This document is the documentation of the findings. It is pushed to GitHub with the source code (see below).

Conclusion

The `procinfo.c` program showed that every call to `printf()`, `getpid()` or `time()` depends on the C Standard Library, which is split between header files in `/usr/include/` and compiled code in `/lib/x86_64-linux-gnu/`. Static linking produced a self-contained 922 KB binary, while dynamic linking produced a small binary that loads `libc.so.6` at run time. Using `readelf` and `ldd`, the ELF header (format and architecture), program headers (memory layout and protections such as a non-executable stack) and dynamic section (required libraries) were inspected.

GitHub repository

File	Description
<code>procinfo.c</code>	Process information program
<code>Lab2.1_Documentation.pdf</code>	This report

Repository link: <https://github.com/KarkiAnish47/os-labs> (folder `lab2.1`)